Lezione 13 - Le map in Java

Introduzione alle Map in Java

Dopo aver analizzato le **Collection**, passiamo ora alle **Map**.

La prima cosa importante da chiarire è che `Map`:

- **non estende l'interfaccia `Collection`.**
- **È un'interfaccia separata.**
  Il motivo è strutturale:
- **mentre le collection gestiscono insiemi di elementi singoli, le map gestiscono coppie chiave-valore.**

L'interfaccia è definita come:

Java

```java
public interface Map<K, V>
```

La presenza di **due parametri generici (`K` e `V`)** evidenzia che ogni elemento della mappa è composto da:

- **una chiave (`K`)**
- **un valore (`V`)**

> **È molto simile al concetto di** dizionario in Python:**
>
> - **una struttura dati che associa una chiave a un valore.**

Generics e tipi primitivi

Come per tutte le strutture generiche in Java, i tipi devono essere **oggetti**, non primitivi.

Quindi:

- `int` → `Integer`
- `char` → `Character`
- `double` → `Double`

- ecc.

Questo perché i generics lavorano esclusivamente con **reference type.**

I metodi principali di `Map`

Essendo `Map` un interfaccia separata dall'interfaccia `Collection` possiede metodi propri, pensati per la gestione delle associazioni chiave-valore.
Le metodi più note sono:

Inserimento di elementi: `put(k,v)`

- **Per inserire coppie chiave-valore in una mappa**

Java

```
put(K key, V value)
```

- Il primo parametro è la **chiave**
- Il secondo parametro è il **valore**

⚠ **Se si inserisce un nuovo valore con una chiave già esistente, la mappa:**

- **non aggiunge una nuova entry**
- **ma sovrascrive il valore precedente associato a quella chiave**

Inoltre, il metodo `put()`:

- **restituisce il vecchio valore associato alla chiave, se esisteva**
- **restituisce `null` se la chiave non era presente**

Questo permette di capire se stiamo effettuando una sovrascrittura.

Recupero di un valore: `get(k)`

- **Restituisce il valore associato a quella chiave**

Java

```
get(K key)
```

Se la chiave non esiste, restituisce `null`.

Per evitare sovrascritture indesiderate è possibile:

- **controllare prima con `containsKey()`**
- **oppure verificare il valore restituito da `get()`**

Verifica di presenza

La `Map` mette a disposizione metodi per controllare l'esistenza di chiavi o valori.

`containsKey(K key)`

Restituisce un `boolean`:

- `true` **se la chiave è presente**
- `false` **altrimenti**

Java

```java
containsKey(k)
```

`containsValue(V value)`

Restituisce un `boolean`:

- `true` **se il valore è presente nella mappa**
- `false` **altrimenti**

Java

```java
containsValue(V)
```

Iterazione su una Map

Le map **non possono essere iterate direttamente con un for-each**, perché, a differenza delle collection, non implementano `Iterable`.

Per iterare, dobbiamo prima ottenere una vista della mappa sotto forma di collection.

Il metodo più importante è:

Java

```
entrySet()
```

Questo metodo restituisce un:

```java
Set<Map.Entry<K,V>>
```

Quindi, concettualmente, una `Map` può essere vista come:

- **un insieme di Entry, dove ogni chiave è univoca.**

🔗 Collegamento con SQL

Una `Map` funziona come una struttura che permette una ricerca **diretta per chiave**, un po' come:

```sql
SELECT * FROM tabella WHERE chiave = valore;
```

La differenza è che nella `Map` la chiave è progettata proprio per rendere l'accesso diretto ed efficiente, senza dover scansionare tutti gli elementi.

Map.Entry

Abbiamo detto che una `Map` è una struttura che associa una chiave a un valore.
Ma concretamente, come viene modellata questa coppia all'interno della mappa?

Qui entra in gioco `Map.Entry`.

`Map.Entry` è:

- **un'interfaccia interna a `Map` che modella esattamente la coppia chiave–valore (`K`, `V`).**
  Ogni elemento contenuto in una mappa non è semplicemente un valore, ma è una `Entry`, cioè un oggetto che contiene:
- **la chiave (`k`)**
- **il valore associato a quella chiave (`V`)**

In questo senso possiamo dire che:

- **una `Map` può essere vista come un insieme di Entry.**

Natura di `Map.Entry`

`Map.Entry` è un'interfaccia, quindi:

- **non può essere istanziata direttamente**
- **ogni implementazione concreta di `Map` fornisce una propria implementazione concreta di `Entry`**

Questo significa che noi non creiamo manualmente le entry:

- **le otteniamo dalla mappa, ad esempio tramite il metodo `entrySet()`.**

I metodi di Map.Entry

Ogni `Entry<K,V>` mette a disposizione alcuni metodi fondamentali.

Java

```java
K getKey()
```

Restituisce la chiave associata a quella entry.

Java

```java
V getValue()
```

Restituisce il valore associato alla chiave.

Java

```java
V setValue(V value)
```

Permette di modificare il valore dell'entry, lasciando invariata la chiave.
Restituisce inoltre il vecchio valore.

> ⚠ **È importante notare che non esiste un metodo `setKey()`.**

La chiave non può essere modificata, perché rappresenta l'identificatore univoco dell'entry all'interno della mappa. Cambiarla significherebbe alterare la struttura interna della `Map`.

Le tre "viste" di una Map

Una mappa può essere osservata da tre prospettive diverse.
Possiamo infatti considerarla come:

- **un insieme di chiavi**
- **una collection di valori**
- **un insieme di coppie chiave–valore**

Queste tre viste sono ottenibili tramite tre metodi distinti:

1. **Set delle chiavi:**

```java
Set<K> keySet();
```

2. **Collection dei valori**

```java
Collection<V> values()
```

3. **Set delle entry (chiave-valore)**

```java
Set<Map.Entry<K,V>> entrySet()
```

Queste non sono copie indipendenti, ma **viste collegate alla mappa**:

- **modificando la mappa, cambiano anche le viste (e viceversa, nei limiti consentiti).**

Quindi la stessa mappa può essere trattata:

- **come un `Set` di chiavi (tutte uniche)**

- **come una `Collection` di valori**
- **come un `Set` di `Entry`**

È importante sottolineare che queste non sono copie indipendenti, ma viste collegate alla mappa originale.
Se la mappa cambia, cambiano anche queste strutture.

Iterazione di una Map

Come anticipato poco sopra, una `Map` non implementa `Iterable`, quindi non è possiamo fare direttamente:

```java
for (...) { }
```

Per poter iterare la mappa dobbiamo prima estrarre una delle sue viste, tipicamente l'insieme delle entry.

Ecco un esempio in sintassi più "classica":

```java
Set entries = personale.entrySet();
Iterator iter = entries.iterator();

while (iter.hasNext()) {
    Map.Entry entry = (Map.Entry) iter.next();
    Object key = entry.getKey();
    Object value = entry.getValue();
}
```

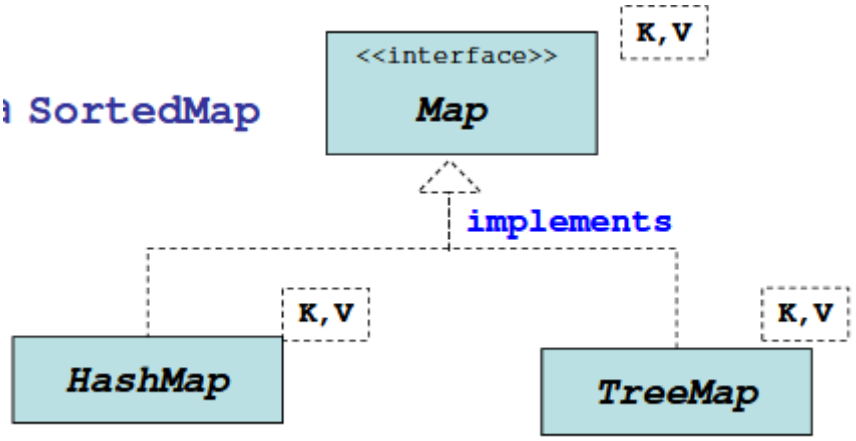
Il ragionamento è il seguente:

1. **Estraggo dalla mappa il `Set` delle entry.**
2. **Ottengo un `Iterator`.**
3. **Scorro il set.**
4. **Per ogni entry recupero chiave e valore.**

Quindi la mappa, di per sé, non è iterabile.
Diventa iterabile nel momento in cui estraiamo una delle sue viste, in particolare `entrySet()`, che è il modo più completo perché permette di accedere contemporaneamente sia alla chiave sia al valore.

Implementazioni di `Map`

Abbiamo visto cos'è l'interfaccia `Map` e quali sono i suoi metodi principali.
Ora passiamo alle **implementazioni concrete**, cioè alle classi che effettivamente realizzano il comportamento della mappa.



Analizzando questa immagine notiamo che le principali classi che implementano `Map` sono:

- `HashMap<K,V>`
- `TreeMap<K,V>`

> ⓘ `TreeMap`, **in particolare, implementa anche l'interfaccia** `SortedMap`**, quindi mantiene le chiavi ordinate.**

> ⚠ **Indipendentemente dall'implementazione scelta, i metodi fondamentali rimangono gli stessi:**

Java

```java
V put(K key, V value)  // inserimento
V get(K key)   // ricerca per chiave (restituisce null se non trova)
boolean containsKey(K key) // verifica presenza chiave
boolean containsValue(V value)// verifica presenza valore
```

Quello che cambia tra le varie implementazioni è **come** vengono gestiti internamente i dati.

Classe `HashMap<K,V>`

La `HashMap` è una delle implementazioni più importanti e utilizzate dell'interfaccia `Map`.
Viene impiegata per la gestione di:

- **mappe non ordinate, cioè strutture in cui non è richiesto alcun ordinamento delle chiavi.**

Si basa su una **hash table**, **quindi l'accesso agli elementi avviene tramite il calcolo dell'hash della chiave.**

Ruolo di `hashCode()` ed `equals()`

Abbiamo visto nella precedente lezione come il ruolo del metodo `hashCode` sia fondamentale per i `Set`.
Anche per una `HashMap` affinché funzioni correttamente, le chiavi degli oggetti inseriti nella mappa devono rispettare una regola fondamentale:

> **Due oggetti che risultano uguali secondo `equals()` devono produrre lo stesso valore di `hashCode()`.**

In altre parole:

- `equals()` **serve a stabilire se due chiavi sono logicamente uguali.**
- `hashCode()` **serve a determinare in quale bucket verrà inserita la chiave.**

Il flusso è il seguente:

1. **Quando inseriamo una chiave, viene calcolato il suo `hashCode()`.**
2. **L'hash determina la posizione nell'array (cioè il bucket).**
3. **Se nel bucket sono già presenti elementi, viene utilizzato `equals()` per distinguere eventuali chiavi duplicate.**

Se non si ridefiniscono correttamente `equals()` e `hashCode()`, la mappa può:

- **non riconoscere chiavi duplicate,**
- **oppure non trovare una chiave già presente.**

Struttura interna: array e bucket

Per ottimizzare le operazioni di inserimento e ricerca, la `HashMap` utilizza internamente:

- **un array**
- **i cui elementi sono delle liste**
- **ogni lista rappresenta un bucket**

Ogni bucket contiene tutte le entry che hanno prodotto lo stesso indice dopo il calcolo dell'hash.

Se due chiavi diverse producono lo stesso indice, si verifica una **collisione**, e le entry vengono inserite nella stessa lista.

Questo meccanismo consente di mantenere mediamente molto veloci le operazioni di:

- `put()`
- `get()`
- `containsKey()`

Capacita iniziale e fattore di carico

La `HashMap` permette di configurare due parametri fondamentali tramite il costruttore:

- **initialCapacity** → **numero iniziale di bucket**
- **loadFactor** → **fattore di carico**
  I valori di default sono:

```java
initialCapacity = 16
loadFactor = 0.75
```

Significato del load factor

Il fattore di carico rappresenta **il rapporto massimo tra**:

```nginx
numero di elementi / numero di bucket
```

Quando questo rapporto supera il valore impostato (di default 0.75), la mappa:

- **aumenta la dimensione dell'array**
- **ricalcola gli hash**

- **redistribuisce le entry nei nuovi bucket**

Questo processo si chiama **rehashing**.

Il load factor è quindi un compromesso tra:

- **uso della memoria**
- **velocità delle operazioni**

Un valore troppo alto aumenta le collisioni.
Un valore troppo basso aumenta lo spreco di memoria.

Analogia della HashMap: lo schedario

Una `HashMap` può essere vista come un **grande schedario da ufficio**.

1. L'array → Lo schedario con i cassetti

La struttura interna della `HashMap` è un **array**.
Possiamo immaginarlo come uno schedario composto da vari **cassetti**.

Ogni cassetto rappresenta un bucket.

Il numero di cassetti iniziali (di default 16) corrisponde alla capacità iniziale della mappa.
2. **L'hash→ Il criterio per scegliere il cassetto**
Quando inseriamo una chiave nella `HashMap`, Java calcola il suo `hashCode()`.

Nell'analogia:

- **l' `hashCode()` è la regola che decide in quale cassetto mettere il fascicolo.**

Non scegliamo il cassetto manualmente:

- **è il valore dell'hash che determina automaticamente dove andrà inserita l'entry.**

3. Il bucket → Il contenuto del cassetto
   Ogni cassetto non contiene un solo fascicolo, ma può contenerne più di uno.

Infatti, **ogni bucket è una lista di elementi.**

Se due chiavi producono lo stesso hash (collisione):

- **finiscono nello stesso cassetto**
- **ma restano comunque distinguibili**

Qui entra in gioco `equals()`.

4. La chiave → Il porta-fascicolo

All'interno di ogni cassetto troviamo dei porta-fascicoli.

Il porta-fascicolo rappresenta la chiave.

Serve per identificare in modo univoco il fascicolo corretto tra quelli presenti nello stesso cassetto.

Quando cerchiamo un elemento:

- **Si calcola l'hash della chiave.**
- **Si apre il cassetto corrispondente.**
- **Se ci sono più fascicoli dentro, si confrontano le etichette (cioè si usa `equals()`).**
- **Si trova quello corretto.**

5. Il valore → Il fascicolo

Il **fascicolo vero e proprio** rappresenta il **valore** associato alla chiave.

La chiave serve solo per organizzare e trovare il fascicolo.
Il valore è l'informazione che ci interessa davvero.

Il fattore di carico → Quando lo schedario si riempie

Immaginiamo ora che i cassetti inizino a riempirsi troppo.

Se ogni cassetto contiene troppi fascicoli:

- **diventa più lento trovare quello giusto.**

Il **load factor (0.75 di default)** stabilisce quando è il momento di:

- **aggiungere nuovi cassetti**
- **riorganizzare tutto lo schedario**

Questo processo è il **rehashing**.

È come comprare un nuovo schedario più grande e ridistribuire tutti i fascicoli nei nuovi cassetti.

Classe `TreeMap`

La `TreeMap` è un'implementazione dell'interfaccia `Map` utilizzata:

- **quando è necessario mantenere le chiavi ordinate.**

A differenza della `HashMap`, che non garantisce alcun ordine, la `TreeMap` organizza automaticamente le chiavi secondo un criterio di ordinamento.

Internamente utilizza una **struttura ad albero bilanciato** (tipicamente un Red-Black Tree).
Questo significa che gli elementi non sono distribuiti tramite hash, ma posizionati secondo una relazione d'ordine.

Ordinamento delle chiavi

Affinché una `TreeMap` possa funzionare correttamente, le chiavi devono essere confrontabili.

Ci sono due possibilità:

Ordinamento naturale

Le chiavi devono implementare l'interfaccia:

Java

```java
Comparable<T>
```

e quindi fornire il metodo:

Java

```java
compareTo(T other)
```

Come abbiamo già visto per le collection `Set` il metodo `compareTo()` stabilisce:

- se un oggetto è minore
- maggiore
- oppure uguale a un altro

Per molti tipi standard questo è già definito:

- wrapper (`Integer` , `Double` , ecc.)
- `String`
  Questi tipi hanno un **ordine naturale** già implementato.

Ordinamento personalizzato

Se si vuole utilizzare un criterio diverso da quello naturale, è possibile fornire un oggetto di tipo:

Java

```java
Comparator<T>
```

Il `Comparator` definisce esternamente la logica di confronto tra le chiavi.

In questo modo possiamo decidere, ad esempio:

- **ordinamento inverso**
- **ordinamento per un campo specifico**
- **ordinamento composto**

Comportamento dell'ordinamento

Un aspetto importante della `TreeMap` è che:

> **Indipendentemente dall'ordine con cui gli elementi vengono inseriti, quando vengono iterati saranno sempre restituiti secondo l'ordine delle chiavi.**

Questo è il punto centrale che distingue la `TreeMap` dalla `HashMap` .

Inoltre poiché la `TreeMap` utilizza una struttura ad albero bilanciato, le operazioni principali hanno complessità:

Sass (SCSS)

```scss
O(log n)
```

In particolare:

- `containsKey()`
- `get()`
- `put()`
- `remove()`

Questo significa che le operazioni sono leggermente più lente rispetto alla `HashMap` (che mediamente lavora in tempo costante O(1)), ma garantiscono l'ordinamento.

Confronto concettuale tra `HahsMap` e `TreeMap`

Possiamo riassumere così:

- `HashMap`
  - Non ordinata
  - Basata su hash table
  - Tempo medio O(1)
- `TreeMap`
  - Ordinata
  - Basata su albero bilanciato
  - Tempo O(log n)

La scelta dipende quindi dal requisito:

- **Se serve solo accesso veloce per chiave** → `HashMap`
- **Se serve mantenere un ordine sulle chiavi** → `TreeMap`